MODULE 14 / LESSON 01

An LLM Application Is More Than a Model Call

An engineered LLM application includes:

DSACADEMY

CORE CONCEPT

Authentication Proves Which Application Is Calling

Provider APIs require authentication, often through an API key or cloud identity. Rules: Authentication identifies a caller. Authorization decides what that caller may do.

keep credentials outside source code and notebooks;	use environment variables or a secret manager;	separate development and production credentials;
grant minimum required permissions and spend limits;	rotate exposed keys;	never log authorization headers;

RUN IT AND INSPECT THE RESULT

Model Selection Starts with Required Capabilities

```
requirements = {
    "structured_output": True,
    "p95_latency_ms": 2000,
    "field_accuracy_min": 0.95,
}
```

OUTPUT

A capability and acceptance contract used to compare candidate models.

Selection depends on the minimum accepted quality and latency requirement.

REASONING SEQUENCE

Streaming Delivers Partial Output Events

Frame

faster time to first visible token;

1

Transform

progressive user feedback;

2

Validate

ability to cancel long generations.

3

Explain

assemble chunks in order;

4

Chunks: "The " -> "answer " -> "is 42." The interface can display text progressively. For JSON, buffer until the complete object arrives, then parse and validate.

CORE CONCEPT

Tokens Drive Context, Latency, and Cost

Provider usage commonly separates: Approximate request cost: inputtokens x inputrate + outputtokens x outputrate

input tokens;

output tokens;

sometimes cached or reasoning tokens.

RUN IT AND INSPECT THE RESULT

Retries Handle Transient Failures, Not Every Failure

```
for attempt in range(3):
    try:
        return call_provider(timeout=10)
    except TransientError:
        sleep(2**attempt + random.random())
raise ProviderUnavailable()
```

OUTPUT

Success within the retry budget or one controlled unavailable error.

With a three-attempt cap and ten-second total budget, the request cannot retry forever.

CORE CONCEPT

A Provider Adapter Normalizes What Can Be Normalized

A provider adapter gives application code one interface for several providers. LiteLLM can normalize common request and response shapes, token usage, exceptions, routing, and fallbacks. Capabilities still differ:

- **tool and schema formats;**
- **supported parameters;**
- **context limits;**
- **multimodal input;**
- **safety behaviour;**
- **streaming events.**

REASONING SEQUENCE

LangChain Composes Models, Prompts, Parsers, and Tools

Frame	Transform	Validate	Explain
chat models;	prompt templates;	structured output;	retrieval;
1	2	3	4

Chain: validated request -> prompt template -> chat model -> structured output model The chain's input is a topic string. Its output is a validated Summary object, not unparsed prose.

RUN IT AND INSPECT THE RESULT

Tracing Records Every Important Boundary

```
with tracer.start_as_current_span("llm_request") as span:
    span.set_attribute("model", model_id)
    result = call_model()
```

OUTPUT

One searchable trace connecting request metadata and nested operation timings.

Total observed span time is approximately 1,287ms, revealing that the model call dominates latency.

QUALITY GATE

MCP Standardizes Tools and Resources

- [x] tools: bounded operations the client may request;
- [x] resources: readable context with stable identifiers;
- [x] prompts: reusable prompt templates where supported.
- [x] resource: approved course handout;

QUALITY GATE

Resilience Needs Explicit Fallback Behaviour

- [x] validation failure;
- [x] context-too-large handling;
- [x] rate-limit and timeout behaviour;
- [x] schema failure;

GUIDED LAB

Guided Lab: Build a Provider-Resilient Service

01

validates request input

02

reads credentials from environment

03

supports two provider configurations

04

checks structured-output capability